AI-powered Virtual Humans

LLM-Powered Cognitive Architectures



Module 1 · Chapter 03
Introduction to LangChain

From Prompt Management to LangChain

The cognitive architecture is implemented in code.

Our code

Prepare context, call the model and use its response.

LangChain

Model interfaces, prompt formatting and connected processing steps.

LangChain helps us write the code that coordinates the loop.

The Building Blocks

Model

Call an LLM through a common interface.

Chain

Pass one output to the next component.

Chat prompt template

Fill variables in a message structure.

Structured output

Return fields the program can use.

Same Model, Different Interface [2]

Compare client creation and the model call.

Provider SDK

```
client = Client(...)
reply = client.chat(
    model="gpt-oss:120b",
    messages=messages,
)
```

LangChain

```
llm = ChatOllama(...)
reply = llm.invoke(request)
```

The Reply Is a Message [1] [2]

Inspect the response object before selecting its text.

```
reply = llm.invoke(request)
reply

reply.content
```



Open 01_langchain.ipynb: 1. First call

A First Chain [3]

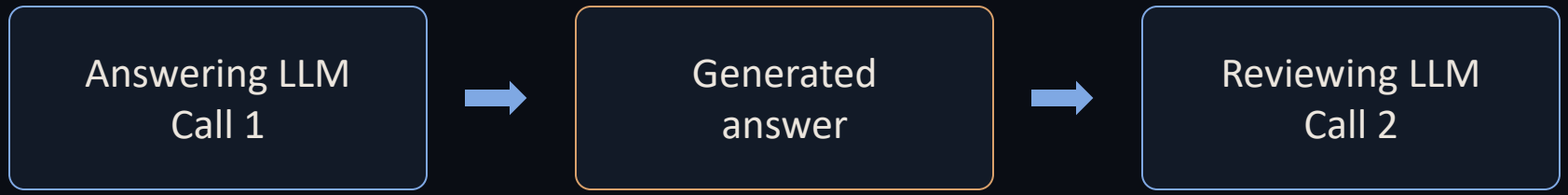
The pipe passes one output to the next component.

```
text_chain = llm | StrOutputParser()
```



StrOutputParser extracts the message text.

Generate an Answer, Then Check It



The second call decides which text the user receives.

An Explicit Checking Rule

```
If the text discusses football, replace it with:
"I am not authorised to discuss football."
Otherwise, return it unchanged.

review_input = guardrail_rules + "\nText:\n" + draft
checked_answer = text_chain.invoke(review_input)
```

Check the generated answer before showing it to the user.

Try the Review Rule

Text about football

The match ended 2-0.
The second model replaces it.

Final text

I am not authorised
to discuss football.

For another topic, the second model keeps the original answer.

Open 01_langchain.ipynb: 2. Chains

A Conversation Can Be a Prompt

The model can continue an unfinished exchange.

```
Visitor: Hello!
Guide: Welcome to the museum.
Visitor: Can you help me explore?
Guide:
```

Examples Show a Character's Style

```
Visitor: Hello!
PippoBot: Ahoy, visitor! Welcome aboard our museum.
Visitor: Can you explain an exhibit?
PippoBot: Arrr, matey! Let us uncover its story together.
Visitor: Can you help me explore?
PippoBot:
```

Chat Models Use Formatted Messages [4]

Chat tuning

The model learns from conversations with roles and message boundaries.

At inference

We supply messages.
The provider applies the model's chat format.

The model still predicts tokens.

Three Message Types [5]

SystemMessage

Instructions:
"You are a pirate museum guide."

HumanMessage

User input:
"Can you help me explore?"

AIMessage

Assistant reply or supplied example:
"Ahoy, visitor!"

A Conversation as Message Objects [5]

```
messages = [
    SystemMessage("You are a pirate museum guide."),
    HumanMessage("Hello!"),
    AIMessage("Ahoy, visitor!"),
    HumanMessage("Can you help me explore?"),
]
for message in messages:
    message.pretty_print()
```

Message Templates: Static and Dynamic Parts [6]

```
system_template = SystemMessagePromptTemplate.from_template(
    "You are {character}. Speak in a {style} style."
)
message = system_template.format(
    character="PippoBot", style="pirate"
)
```

Fixed text stays the same; braces mark values supplied at each call.

One Dictionary Fills the Chat Prompt [6]

```
chat_prompt = ChatPromptTemplate.from_messages([
    system_template, human_template,
])
values = {"character": "PippoBot", "style": "pirate",
          "user_input": "Can you help me explore?"}

chat_chain = chat_prompt | text_chain
chat_chain.invoke(values)
```

The dictionary fills the variables in all message templates.

The Same Guardrail with Chat Templates

System: fixed rules

If the text discusses football,
replace it with the refusal.
Otherwise, keep it unchanged.

Human: dynamic text

{text}

The answer from the first LLM.

```
review_chain.invoke({"text": "The football match ended 2-0."})
```

Exercise: Style and Current Time

Tune the prompt

Choose how the bot should speak. Message templates fill style and time.

Try on the agent

Copy the style into STYLE_PROMPT. Restart the server and ask: "What time is it?"

Open 01_langchain.ipynb: 3. Conversations and chat prompts

Structured Output [7]

Sentiment analysis: return a category the program can use.

Review

I loved the exhibition!

```
{
  "sentiment": "POSITIVE"
}
```

A schema specifies the fields and their types.

Format Instructions for Every Schema

Add schema fields and allowed values to the system message.

```
format_instructions = json.dumps(Sentiment.model_json_schema())
schema_template = SystemMessagePromptTemplate.from_template(
    "{format_instructions}")
schema_template.format(format_instructions=format_instructions)
```

Then classify an email as SUPPORT, SALES or SPAM.

Open 02_structured_output.ipynb: 1. Sentiment, format instructions and email

Several Labels in One Response

Requests

Where is the cafe?
When do you close?
How much is a ticket?



```
labels = [
    "DIRECTIONS",
    "HOURS",
    "TICKETS",
]
```

One list of requests, one model call, labels in the same order.

Exercise: Tune the Classifier Prompt

15 requests

One dataset, one model call.
Inspect the wrong predictions.
Tune prompt and description.

One score

Keep labels out of the input.
Rerun the same evaluation.
Aim for the highest accuracy.

The code is ready. Tune the prompt and field description.

Inspect Errors and Improve Accuracy

Accuracy = correct predictions / number of requests

Inspect the errors, tune the prompt, then classify the requests again.

Open 02_structured_output.ipynb: 2. Tune a classifier prompt

A Structured Reply Controls the Avatar

Return the fields expected by the server.

```
{
  "speech_text": "Welcome!",
  "emotion": "HAPPY",
  "expression": null,
  "gesture": "handup"
}
```

Avatar behaviour

Speak the text.
Set emotion and expression.
Perform the gesture.

Try on the Structured Agent

Reuse the same pattern

ModelReply defines the fields.
System supplies its schema.
Human supplies the request.

Try on the agent

Select the structured agent.
Ask for a happy welcome
and a raised hand.

Open 02_structured_output.ipynb: 3. A response that controls the avatar

Exercise: Short-Term Conversation Memory

Earlier turn

"My name is Lea.
My favourite colour is green."

Later question

"What is my name
and my favourite colour?"

Keep human and AI messages in a list and resend them on every call.
Build your own memory agent from the naive agent, then try both turns.

Open 02_structured_output.ipynb: 4. Short-term conversation memory

Final Exercise: Memory and Avatar Control

Your memory agent

A list of human and AI messages, resent on every call.

The structured agent

speech_text, emotion, expression and gesture.

Combine both: the avatar remembers you and keeps moving.

Write the code yourself; the file lists the steps.

Open langchain_structured_memory_agent.py

References

[1] LangChain models

Common interfaces and composition

[2] ChatOllama

Provider integration and model calls

[3] RunnableSequence

Compose components with the pipe operator

[4] Chat templates

Conversations, tokens and model-specific formats

[5] LangChain messages

System, human and AI message objects

[6] ChatPromptTemplate

Build and fill reusable message templates

[7] ChatOllama structured output

Schema wrapper and provider-compatible methods

[8] Ollama structured outputs

Cloud support and native schema limitations